
Rapport Technique

Services Conducteurs & Maintenance

Microservices NestJS et FastAPI — API Gateway, Kafka et Keycloak

Système de Gestion de Flotte de Véhicules

Master 1 GIL — 2025/2026

Informations du projet

Auteurs :	Kherbachi Rima & Mouhoubi Leiticia
Formation :	Master 1 GIL
Université :	Université de Rouen Normandie
Encadrants :	Lydia et Luc
Année :	2025 – 2026
Semaine documentée :	4

Mars 2026

Table des matières

1	Introduction et Architecture Multi-Services	3
I	Service Conducteur (NestJS / TypeScript)	4
2	Architecture du Service Conducteur	5
2.1	Architecture NestJS en couches	5
2.2	Modules NestJS	5
3	Bibliothèques et Dépendances (NestJS)	6
4	Modèle de Données (TypeORM)	7
4.1	Entité Conducteur	7
4.2	Entité Assignment	8
5	API REST et Sécurité Keycloak (NestJS)	9
5.1	Endpoints REST	9
5.2	Sécurité JWT Keycloak	10
5.3	Logique métier : Validation du permis	10
6	Intégration Apache Kafka (Saga Choréographiée)	12
6.1	Diagramme de la Saga	12
6.2	Producer Kafka	12
6.3	Consumer Kafka	13
7	Tests et Couverture	14
7.1	Stratégie de tests	14
7.2	Résultat de la couverture	14
7.3	Validation de l'API avec Bruno / Postman	15
7.3.1	Tests CRUD des conducteurs	15
7.3.2	Validation du permis	17
7.3.3	Gestion des assignments	17
7.3.4	Suppression d'un conducteur par identifiant	19
7.3.5	Health checks	20
7.4	Validation via l'API Gateway Apollo (GraphQL)	20

II	Service Maintenance (Python / FastAPI)	21
8	Architecture du Service Maintenance	22
8.1	Architecture générale	22
8.2	Modélisation et Logique Métier	22
9	Bibliothèques et Dépendances (FastAPI)	23
10	Modèle de Données (SQLAlchemy)	24
11	API REST et Sécurité Keycloak (FastAPI)	25
11.1	Endpoints REST	25
11.2	Sécurité JWT	25
12	Intégration Apache Kafka (FastAPI)	27
12.1	Consommateur	27
13	Tests et Validation Client (FastAPI)	28
13.1	Validation de l'API avec Bruno	28
13.2	Validation via l'API Gateway Apollo (GraphQL)	29
13.3	Couverture des tests Pytest	30
	Conclusion	31

Chapitre 1

Introduction et Architecture Multi-Services

La semaine 4 de notre projet est marquée par le développement et l'intégration en parallèle de deux nouveaux microservices au sein de notre flotte :

- **Le service conducteur-service** : Développé en **NestJS / TypeScript** par Rima Kherbachi, responsable de la gestion des profils conducteurs, de leurs permis et de leurs affectations de véhicules.
- **Le service maintenance-service** : Développé en **Python (FastAPI)** par Leïticia Mouhoubi, responsable du cycle de vie des interventions mécaniques.

L'utilisation de technologies hétérogènes (NestJS et Python) démontre la flexibilité de notre architecture orientée microservices, unifiée par notre *API Gateway Apollo GraphQL* et notre bus de messages *Apache Kafka*.

lightblue Service	Technologie	Port	Responsabilité
conducteur-service	NestJS / TypeScript	3000	Gestion conducteurs, permis, assignations
maintenance-service	Python / FastAPI	8002	Gestion des interventions mécaniques

TABLE 1.1 – Vue d'ensemble des deux services développés en semaine 4

Première partie

Service Conducteur (NestJS / TypeScript)

Chapitre 2

Architecture du Service Conducteur

2.1 Architecture NestJS en couches

Le service suit l'architecture modulaire imposée par NestJS, organisée en couches strictement séparées :

1. **Couche Contrôleur** (`conducteurs.controller.ts`, `assignments.controller.ts`) : Expose les routes REST, gère la validation des DTOs via `class-validator`.
2. **Couche Service** (`conducteurs.service.ts`, `assignments.service.ts`) : Contient toute la logique métier, y compris la validation des permis.
3. **Couche Base de données** (`*.entity.ts`) : Définit les entités TypeORM mappées vers PostgreSQL.
4. **Couche Sécurité** (`jwt.strategy.ts`, `jwt-auth.guard.ts`, `roles.guard.ts`) : Validation JWT Keycloak et contrôle d'accès basé sur les rôles (RBAC).
5. **Couche Événementielle** (`kafka.producer.ts`, `kafka.consumer.ts`) : Publie et consomme les événements Kafka (Saga Choréographiée).
6. **Observabilité** (`tracing.ts`) : Traces distribuées via OpenTelemetry / Jaeger et métriques Prometheus.

2.2 Modules NestJS

Le service est décomposé en quatre modules indépendants :

lightblue Module	Responsabilité
ConducteursModule	CRUD conducteurs, validation permis
AssignationsModule	Cycle de vie des assignations conducteur-véhicule
KafkaModule	Producer et Consumer Kafka (Saga)
AuthModule	JWT Strategy, Guards, décorateurs RBAC

TABLE 2.1 – Modules NestJS du conducteur-service

Chapitre 3

Bibliothèques et Dépendances (NestJS)

lightblue thèque	Biblio-	Package npm	Rôle et Justification
NestJS Core		@nestjs/common, @nestjs/core	Framework principal, injection de dépendances, décorateurs.
TypeORM & BD		@nestjs/typeorm, typeorm, pg	ORM TypeScript pour mapper les entités vers PostgreSQL.
Sécurité		@nestjs/passport, passport-jwt, jwks-rsa	Validation des tokens JWT Keycloak via les clés JWKS publiques.
Kafka		kafkajs	Client Kafka natif TypeScript pour la Saga choréographiée.
Validation DTOs		class-validator, class-transformer	Validation automatique des corps de requête via décorateurs.
Observabilité		@opentelemetry/sdk-node, @opentelemetry/exporter-jaeger	Traces distribuées et métriques Prometheus.
Documentation		@nestjs/swagger	Génération automatique de la spec OpenAPI (Swagger UI).
Tests		jest, ts-jest, @nestjs/testing	Tests unitaires avec mocks TypeScript, couverture Istanbul.

TABLE 3.1 – Dépendances du conducteur-service (`package.json`)

Chapitre 4

Modèle de Données (TypeORM)

4.1 Entité Conducteur

L'entité `Conducteur` est mappée sur la table `conducteur` en PostgreSQL. Elle stocke les informations du profil conducteur et la validité de son permis.

```
1 export enum CategoriePermis { A = 'A', B = 'B', C = 'C', D = 'D' }
2
3 @Entity('conducteur')
4 export class Conducteur {
5     @PrimaryGeneratedColumn('uuid', { name: 'id_conducteur' })
6     idConducteur: string;
7
8     @Column({ name: 'keycloak_user_id', unique: true, nullable: true
9         })
10    keycloakUserId: string;
11
12    @Column({ length: 100 })
13    nom: string;
14
15    @Column({ length: 100 })
16    prenom: string;
17
18    @Column({ unique: true })
19    email: string;
20
21    @Column({ name: 'numero_permis', length: 50, unique: true })
22    numeroPermis: string;
23
24    @Column({ type: 'enum', enum: CategoriePermis, array: true })
25    categorie: CategoriePermis[];
26
27    @Column({ type: 'date', name: 'date_validite_permis' })
28    dateValiditePermis: Date;
29
30    @Column({ default: true })
31    actif: boolean;    // Soft delete
```

```
32 @CreateDateColumn({ name: 'created_at', type: 'timestampz' })
33 createdAt: Date;
34
35 @OneToMany(() => Assignment, (a) => a.conducteur)
36 assignments: Assignment[];
37 }
```

Listing 4.1 – Définition de l'entité Conducteur (conducteur.entity.ts)

4.2 Entité Assignment

L'entité Assignment modélise l'affectation d'un conducteur à un véhicule pour une période donnée.

```
1 export enum StatutAssignment {
2   PLANIFIEE = 'planifiee',
3   EN_COURS = 'en_cours',
4   TERMINEE = 'terminee',
5   ANNULEE = 'annulee',
6 }
7
8 @Entity('assignment')
9 @Check('"date_retour" IS NULL OR "date_retour" > "date_depart"')
10 export class Assignment {
11   @PrimaryGeneratedColumn('uuid', { name: 'id_assignment' })
12   idAssignment: string;
13
14   @Column({ name: 'vehicule_id', type: 'uuid' })
15   vehiculeId: string;
16
17   @Column({ name: 'conducteur_id', type: 'uuid', nullable: true })
18   conducteurId: string;
19
20   @Column({ name: 'date_depart', type: 'timestampz' })
21   dateDepart: Date;
22
23   @Column({ name: 'date_retour', type: 'timestampz', nullable:
24     true })
25   dateRetour: Date;
26
27   @Column({ type: 'enum', enum: StatutAssignment,
28     default: StatutAssignment.EN_COURS })
29   statut: StatutAssignment;
30
31   @ManyToOne(() => Conducteur, (c) => c.assignations)
32   @JoinColumn({ name: 'conducteur_id' })
33   conducteur: Conducteur;
34 }
```

Listing 4.2 – Définition de l'entité Assignment (assignment.entity.ts)

Chapitre 5

API REST et Sécurité Keycloak (NestJS)

5.1 Endpoints REST

lightblue Méth.	URL	Rôles requis	Description
GET	/api/conducteurs	tous	Lister les conducteurs actifs
POST	/api/conducteurs	admin, manager	Créer un conducteur
GET	/api/conducteurs/:id	tous	Détail d'un conducteur
PUT	/api/conducteurs/:id	admin, manager	Mettre à jour
DELETE	/api/conducteurs/:id	admin	Soft delete
GET	/api/conducteurs/:id/permis/valider	tous	Vérifier la validité du permis
GET	/api/conducteurs/:id/assignments	tous	Historique des assignments
GET	/api/assignments	tous	Lister les assignments
POST	/api/assignments	admin, manager	Créer une assignment
GET	/api/assignments/:id	tous	Détail assignment
PATCH	/api/assignments/:id/terminer	admin, manager, technicien	Terminer une assignment
PATCH	/api/assignments/:id/annuler	admin, manager	Annuler une assignment
GET	/health	public	Liveness probe (TypeORM ping)

TABLE 5.1 – Endpoints REST du conducteur-service

5.2 Sécurité JWT Keycloak

La sécurité repose sur deux guards appliqués globalement :

- `JwtAuthGuard` : Hérite de `AuthGuard('jwt')`. Bypass automatique des routes annotées `@Public()`. Rejette tout token invalide ou absent avec HTTP 401.
- `RolesGuard` : Vérifie les rôles extraits du token JWT (via `realm_access.roles` et `resource_access.fleet-app.roles`). Rejette les accès non autorisés avec HTTP 403.

```

1 @Injectable()
2 export class JwtStrategy extends PassportStrategy(Strategy) {
3   constructor(private readonly configService: ConfigService) {
4     const jwksUri = configService.get<string>('keycloak.jwksUri');
5     const issuer = configService.get<string>('keycloak.issuer');
6     super({
7       secretOrKeyProvider: passportJwtSecret({
8         cache: true, rateLimit: true,
9         jwksRequestsPerMinute: 5, jwksUri,
10      }),
11      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
12      audience: 'account', issuer, algorithms: ['RS256'],
13    });
14  }
15
16  async validate(payload: any) {
17    const roles = payload?.realm_access?.roles || [];
18    const resourceRoles =
19      payload?.resource_access?.['fleet-app']?.roles || [];
20    return {
21      userId: payload.sub,
22      username: payload.preferred_username,
23      email: payload.email,
24      roles: [...roles, ...resourceRoles],
25    };
26  }
27 }

```

Listing 5.1 – Stratégie JWT Keycloak (jwt.strategy.ts)

5.3 Logique métier : Validation du permis

Avant toute création d'assignation, le service vérifie automatiquement que le permis du conducteur est valide :

```

1 async validerPermis(id: string): Promise<PermisValidationResult> {
2   const conducteur = await this.findOne(id);
3   const now = new Date();
4   const diffMs = new Date(conducteur.dateValiditePermis).getTime()

```

```
5         - now.getTime();
6     const joursRestants = Math.floor(diffMs / (1000 * 60 * 60 * 24));
7     const valide = diffMs > 0;
8
9     return {
10         valide,
11         conducteurId: id,
12         numeroPermis: conducteur.numeroPermis,
13         dateValidite: conducteur.dateValiditePermis,
14         joursRestants,
15         message: valide
16             ? 'Permis valide encore ${joursRestants} jour(s)'
17             : 'Permis expire depuis ${Math.abs(joursRestants)} jour(s)',
18     };
19 }
```

Listing 5.2 – Validation métier du permis (conducteurs.service.ts)

Chapitre 6

Intégration Apache Kafka (Saga Choréographiée)

6.1 Diagramme de la Saga

La création d'une assignation déclenche une chorégraphie Kafka entre le service conducteur et le service véhicule :

1. **conducteur-service** publie `AssignationDemandee` sur `fleet.conducteurs.assignation`
2. **vehicule-service** consomme l'événement et répond sur `fleet.vehicules.events`
3. **conducteur-service** consomme la réponse :
 - `VehiculeAssigneAvecSucces` $\Rightarrow$ statut `EN_COURS`
 - `EchecAssignationVehicule` $\Rightarrow$ statut `ANNULEE`

6.2 Producer Kafka

```
1 async publishAssignationDemandee(  
2   event: AssignationDemandeeEvent  
3 ): Promise<void> {  
4   const topic = this.configService.get<string>(  
5     'kafka.topics.assignationDemandee'  
6   );  
7   const span = getTracer().startSpan('kafka.publish.  
8     AssignationDemandee');  
9   try {  
10    if (!this.connected) await this.connect();  
11    await this.producer.send({  
12      topic,  
13      compression: CompressionTypes.GZIP,  
14      messages: [{  
15        key: event.assignmentId,  
16        value: JSON.stringify(event),  
17        headers: {  
18          eventType: event.eventType,  
19          source: 'conducteur-service',
```

```
19         timestamp: event.timestamp,
20     },
21     ]],
22 });
23 span.setStatus({ code: SpanStatusCode.OK });
24 } catch (err) {
25     span.setStatus({ code: SpanStatusCode.ERROR, message: err.
26         message });
27     throw err;
28 } finally { span.end(); }
```

Listing 6.1 – Publication de l'événement AssignmentDemandee (kafka.producer.ts)

6.3 Consumer Kafka

```
1 private async handleMessage(payload: EachMessagePayload): Promise<
2     void> {
3     const event: VehiculeAssigneEvent =
4         JSON.parse(payload.message.value.toString());
5
6     if (event.eventType === 'VehiculeAssigneAvecSucces') {
7         const assignment = await this.assignmentRepository.findOne(
8             { where: { idAssignment: event.assignmentId } }
9         );
10        if (assignment) {
11            assignment.statut = StatutAssignment.EN_COURS;
12            await this.assignmentRepository.save(assignment);
13        }
14    } else if (event.eventType === 'EchecAssignmentVehicule') {
15        const assignment = await this.assignmentRepository.findOne(
16            { where: { idAssignment: event.assignmentId } }
17        );
18        if (assignment) {
19            assignment.statut = StatutAssignment.ANNULEE;
20            await this.assignmentRepository.save(assignment);
21        }
22    }
23 }
```

Listing 6.2 – Traitement des réponses véhicule (kafka.consumer.ts)

Chapitre 7

Tests et Couverture

7.1 Stratégie de tests

Les tests unitaires sont colocalisés avec le code source (*.spec.ts). Chaque couche est testée indépendamment via des mocks TypeScript :

lightblue Fichier test	Mock principal	Scénarios couverts
conducteurs.service.spec.ts	Repository TypeORM	CRUD, validation permis, conflits
conducteurs.controller.spec.ts	ConducteursService	Routes, codes HTTP, DTOs
assignments.service.spec.ts	Repository TypeORM	Cycle de vie, transitions de statut
assignments.controller.spec.ts	AssignationsService	Routes, terminer, annuler
kafka.producer.spec.ts	KafkaJS (inline mock)	Connexion, publication, erreurs
kafka.consumer.spec.ts	KafkaJS + Repository	Saga succès, échec, messages nuls
jwt.strategy.spec.ts	PassportStrategy mock	Extraction rôles realm + resource
roles.guard.spec.ts	Reflector	Autorisation, ForbiddenException
jwt-auth.guard.spec.ts	AuthGuard mock	Routes publiques, token invalide
health.controller.spec.ts	HealthCheckService	Liveness probe, ping DB
configuration.spec.ts	process.env	Valeurs par défaut, variables env

TABLE 7.1 – Couverture des tests unitaires Jest du conducteur-service

7.2 Résultat de la couverture

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	95.84	80.82	92.18	95.98	

FIGURE 7.1 – Résultats Jest — 78 tests passés, couverture > 80%

La configuration Jest exclut les fichiers non testables en unité (`main.ts`, `*.module.ts`, `tracing.ts`) pour ne mesurer que la logique métier réelle.

7.3 Validation de l'API avec Bruno / Postman

Afin de valider le bon fonctionnement des endpoints REST du service conducteur, des tests manuels ont été réalisés à l'aide des outils **Bruno** et **Postman**. Ces outils permettent d'envoyer des requêtes HTTP et de vérifier les réponses du serveur en conditions réelles.

7.3.1 Tests CRUD des conducteurs

Les opérations CRUD complètes ont été testées sur la ressource `conducteurs`.

```
1 POST http://localhost:3001/api/conducteurs
2 Content-Type: application/json
3
4 {
5   "nom": "Martin",
6   "prenom": "Sophie",
7   "email": "sophie.martin@test.com",
8   "numeroPermis": "XY-987654",
9   "categorie": ["B", "C"],
10  "dateValiditePermis": "2032-06-15"
11 }
```

Listing 7.1 – Création d'un conducteur (POST)

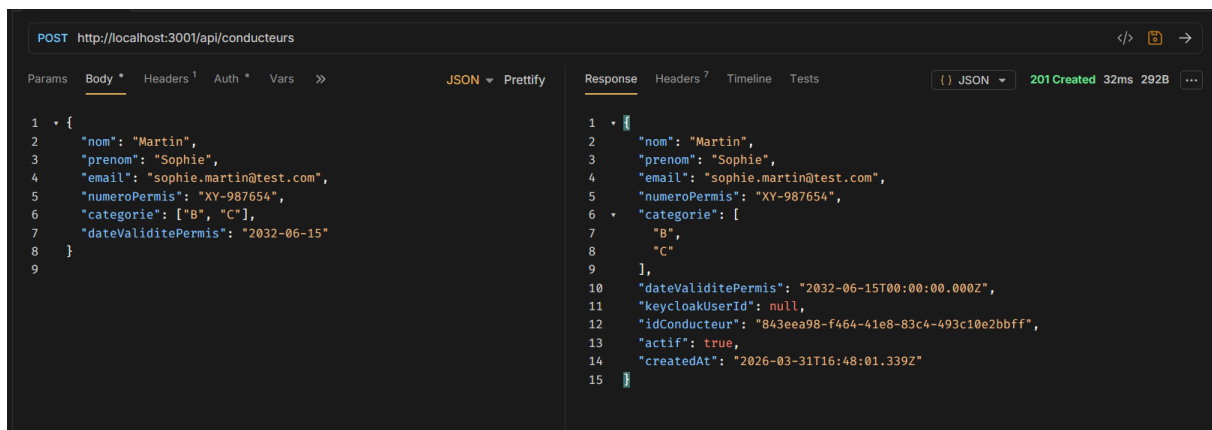


FIGURE 7.2 – Test de création d'un conducteur avec Bruno / Postman

```
1 GET http://localhost:3001/api/conducteurs
```

Listing 7.2 – Consultation de tous les conducteurs (GET)

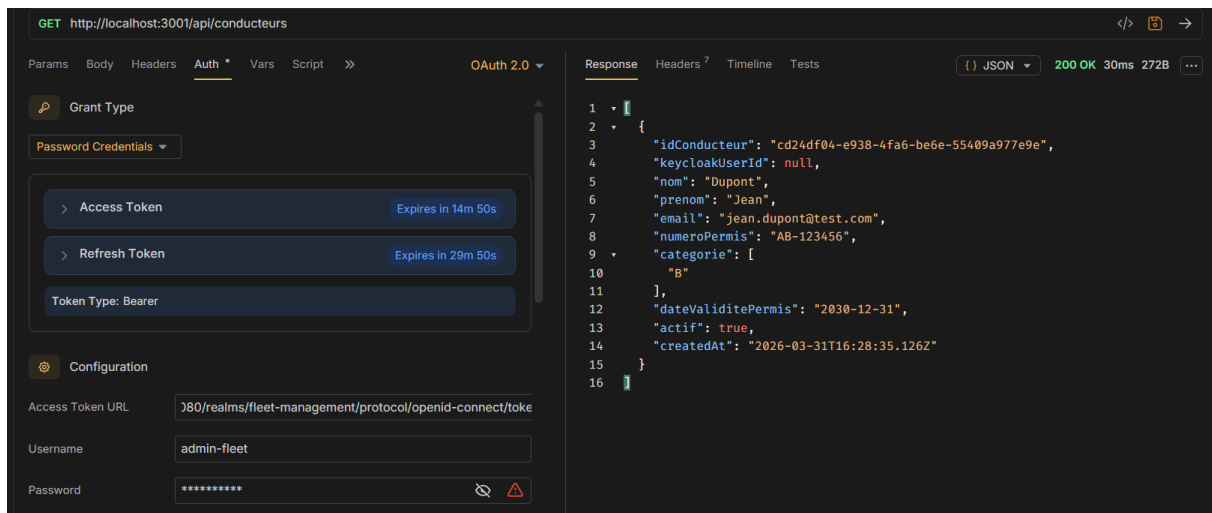


FIGURE 7.3 – Récupération de la liste des conducteurs

```
1 PUT http://localhost:3001/api/conducteurs/{id}
2 Content-Type: application/json
3
4 {
5   "nom": "lolo",
6   "prenom": "Jean (Modifié)",
7   "email": "jean.lolo@test.com",
8   "numeroPermis": "AB-152456",
9   "categorie": ["B", "C"],
10  "dateValiditePermis": "2035-12-31"
11 }
```

Listing 7.3 – Modification d'un conducteur (PUT)

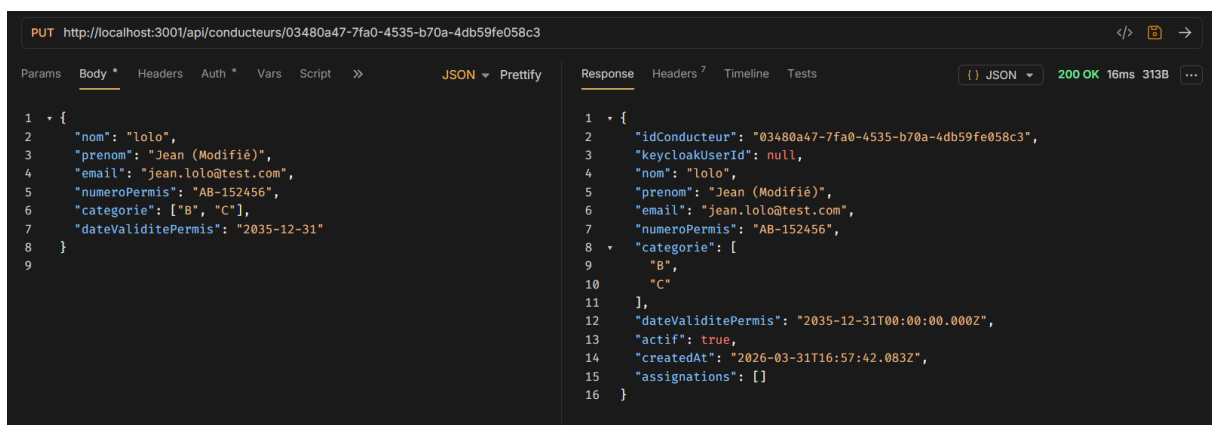


FIGURE 7.4 – Modification d'un conducteur existant

```
1 DELETE http://localhost:3001/api/conducteurs/{id}
```

Listing 7.4 – Suppression d'un conducteur (DELETE)

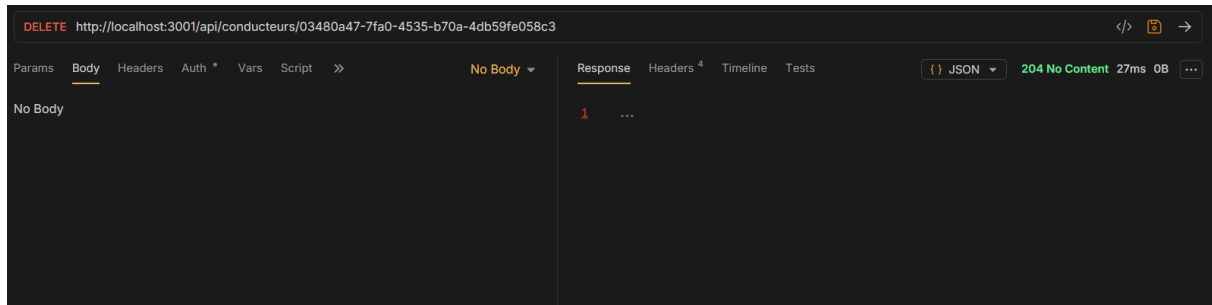


FIGURE 7.5 – Suppression d'un conducteur

7.3.2 Validation du permis

Un endpoint spécifique permet de vérifier la validité d'un permis de conduire ainsi que le nombre de jours restants avant expiration.

```
1 GET http://localhost:3001/api/conducteurs/{id}/permis/valider
```

Listing 7.5 – Vérification de la validité du permis

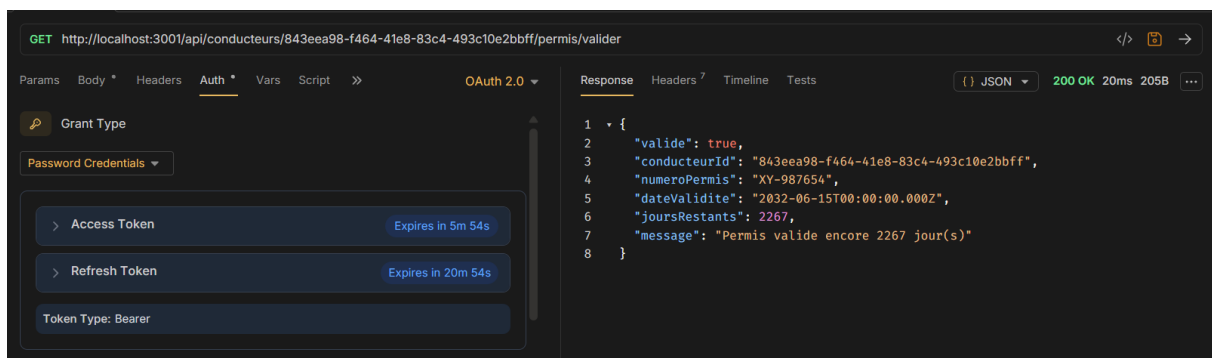


FIGURE 7.6 – Vérification de la validité du permis d'un conducteur

7.3.3 Gestion des assignations

Les tests suivants permettent de valider la gestion des assignations entre un conducteur et un véhicule.

```
1 POST http://localhost:3001/api/assignations
2 Content-Type: application/json
3
4 {
```

```
5  "vehiculeId": "550e8400-e29b-41d4-a716-446655440022",
6  "conducteurId": "03480a47-7fa0-4535-b70a-4db59fe058c3",
7  "dateDepart": "2026-04-01T08:00:00Z"
8  }
```

Listing 7.6 – Création d'une assignation conducteur-véhicule

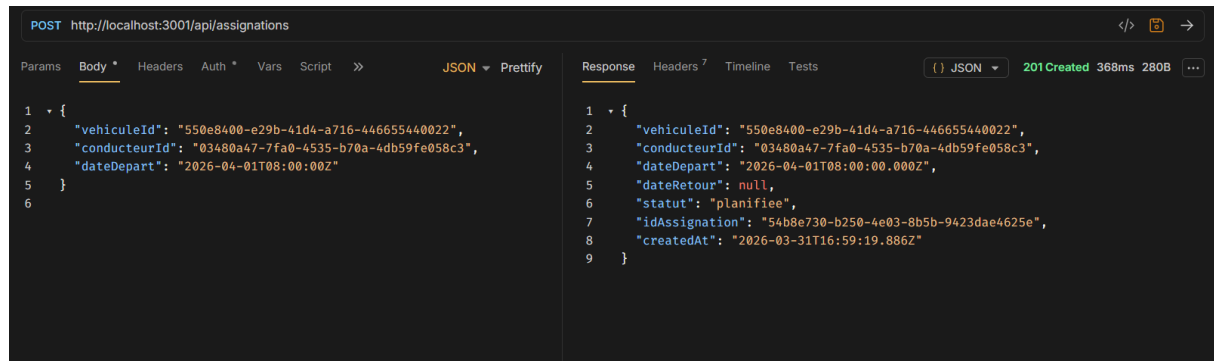


FIGURE 7.7 – Création d'une assignation

```
1 GET http://localhost:3001/api/assignments
```

Listing 7.7 – Consultation de toutes les assignations

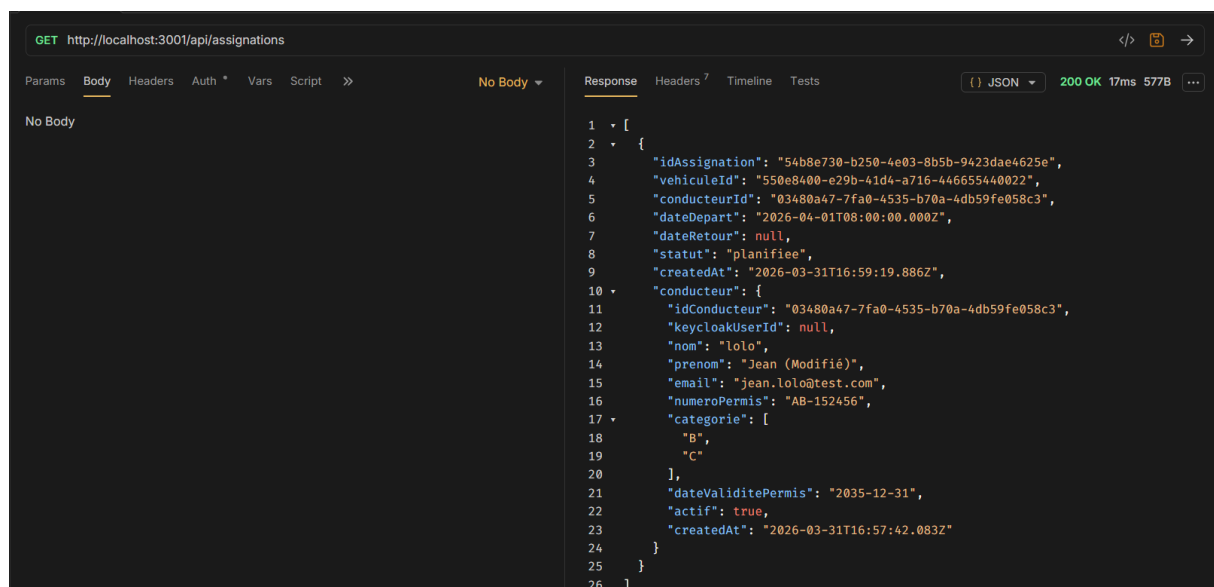
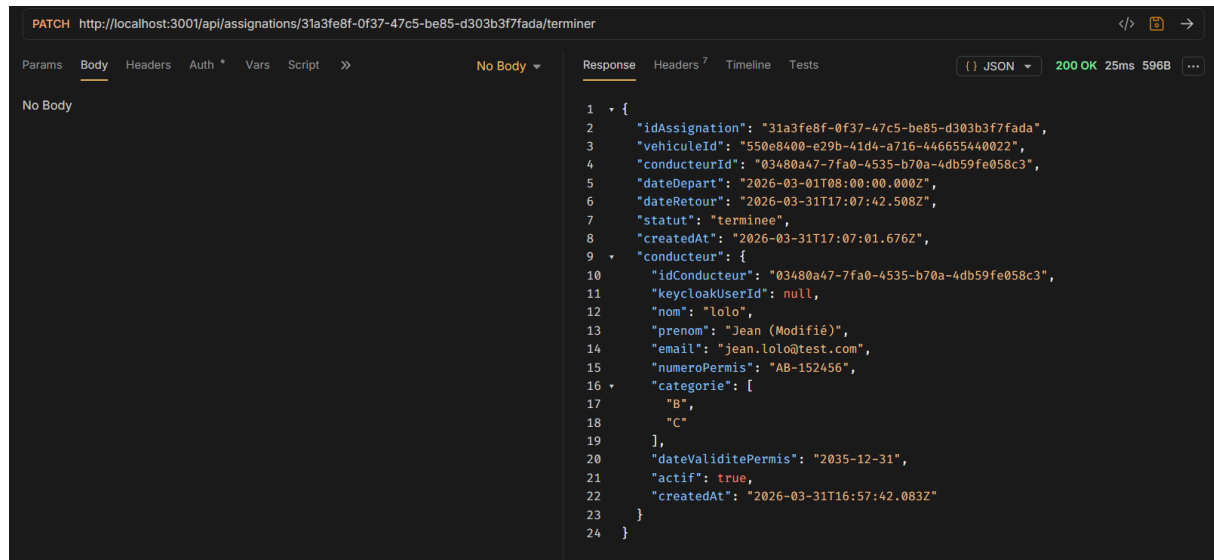


FIGURE 7.8 – Récupération de la liste des assignations

```
1 PATCH http://localhost:3001/api/assignments/{id}/terminer
```

Listing 7.8 – Terminaison d’une assignation

FIGURE 7.9 – Changement du statut d’une assignation en `terminee`

7.3.4 Suppression d’un conducteur par identifiant

Un test supplémentaire a également été réalisé pour vérifier la suppression d’un conducteur précis via son identifiant.

```
1 DELETE http://localhost:3001/api/conducteurs/{id}
```

Listing 7.9 – Suppression ciblée d’un conducteur

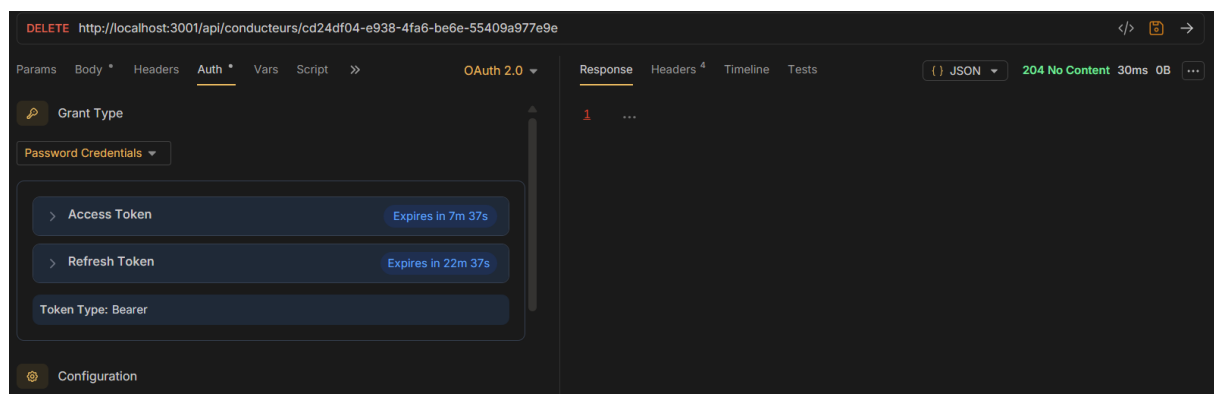


FIGURE 7.10 – Suppression d’un conducteur par identifiant

7.3.5 Health checks

Des endpoints de supervision permettent de vérifier que le service est bien disponible et opérationnel.

```
1 GET http://localhost:3001/  
2 GET http://localhost:3001/health
```

Listing 7.10 – Vérification de l'état du service

7.4 Validation via l'API Gateway Apollo (GraphQL)

Le service conducteur est également testé via l'**API Gateway Apollo**, permettant de valider son intégration dans l'architecture globale du système.

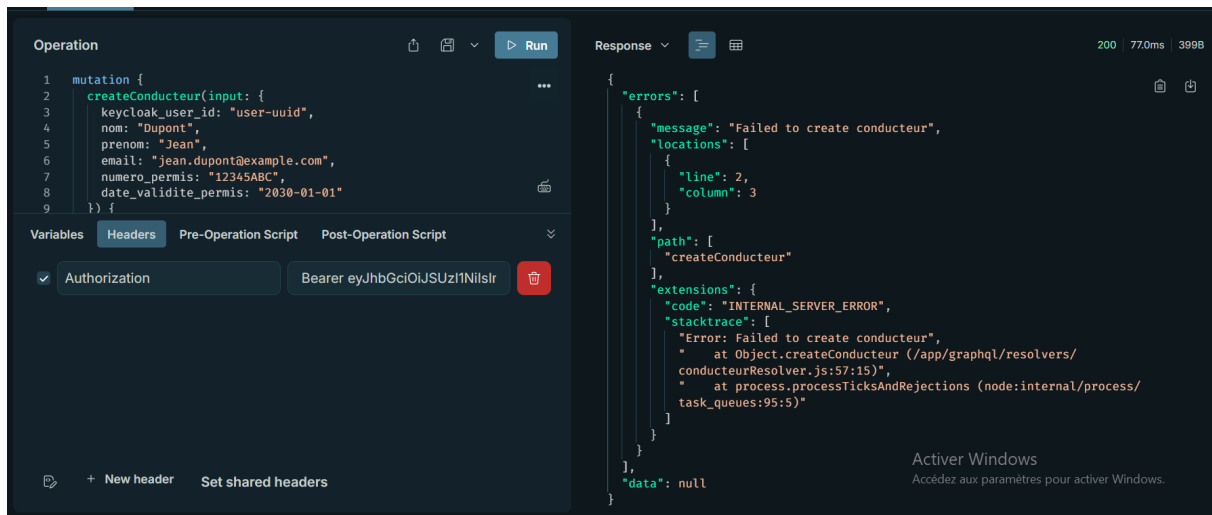


FIGURE 7.11 – Test d'une requête GraphQL via Apollo Sandbox

Deuxième partie

Service Maintenance (Python / FastAPI)

Chapitre 8

Architecture du Service Maintenance

8.1 Architecture générale

Le service suit une approche *Clean Architecture* adaptée pour Python :

1. **Couche Routeur** (`routers.py`) : Expose l'API REST FastAPI et gère les requêtes entrantes.
2. **Couche Service** (`crud.py`) : Contient la logique métier, notamment les règles de transition d'état des interventions et la gestion des coûts.
3. **Couche Base de données** (`database.py`, `models.py`) : Gère la persistance des données avec l'ORM SQLAlchemy.
4. **Couche Événementielle** (`producer.py`, `consumer.py`) : Assure la communication asynchrone via Kafka (consommation des mises à jour de véhicules et publication des fins d'interventions).

Note : La couche de sécurité (validation JWT Keycloak) est implémentée comme un module transverse indépendant afin de ne pas coupler le cœur métier du service à l'infrastructure d'authentification.

8.2 Modélisation et Logique Métier

Au lieu d'un diagramme de classes complexe, la structure du service s'articule autour de deux entités clés qui garantissent l'autonomie du microservice :

- **VehiculeLocal** : Cette classe sert de cache local pour les véhicules. Elle stocke les informations essentielles (ID, immatriculation, statut actuel) synchronisées en temps réel via Kafka depuis le service Véhicule. Cela permet d'effectuer des calculs de maintenance et de valider la disponibilité sans dépendre d'appels API synchrones vers d'autres services.
- **Intervention** : C'est le cœur du domaine. Chaque instance représente une opération de maintenance (révision, réparation, contrôle technique). Elle suit un cycle de vie strict : *planifiée*, *en cours*, puis *terminée*. Les attributs incluent le technicien responsable, les dates prévues et effectives, ainsi que le coût de l'intervention.

La couche `crud.py` orchestre ces entités en appliquant les règles suivantes :

- Une intervention ne peut être annulée que si elle est encore au statut "planifiée".

- La clôture d’une intervention déclenche automatiquement un événement Kafka pour informer le reste du système du nouveau statut du véhicule et du coût engendré.

Chapitre 9

Bibliothèques et Dépendances (FastAPI)

lightblue thèque	Biblio-	Package Pip	Rôle et Justification
FastAPI		<code>fastapi</code> , <code>uvicorn</code>	Framework Web ultra-rapide basé sur Starlette. Support natif de l’asynchrone et génération automatique OpenAPI.
SQLAlchemy & BD		<code>sqlalchemy</code> , <code>psycopg2-binary</code>	ORM synchrone robuste. Standard de l’industrie pour mapper nos entités vers PostgreSQL.
Validation & DTOs		<code>pydantic</code>	Assure la validation stricte des types des requêtes entrantes/sortantes.
Kafka		<code>confluent-kafka</code>	Client C-optimisé très performant pour ingérer les événements liés aux véhicules.
Sécurité Keycloak		<code>python-jose</code>	Permet de télécharger dynamiquement les clés JWKS de Keycloak et de vérifier la signature RSA256 du JWT.

TABLE 9.1 – Dépendances de production FastAPI (`requirements.txt`)

Chapitre 10

Modèle de Données (SQLAlchemy)

```
1 from sqlalchemy import Column, String, Float, DateTime, Enum,
   ForeignKey
2 from sqlalchemy.dialects.postgresql import UUID
3 import uuid, datetime, enum
4 from .database import Base
5
6 class StatutIntervention(str, enum.Enum):
7     PLANIFIEE = "planifiee"
8     EN_COURS  = "en_cours"
9     TERMINEE  = "terminee"
10
11 class Intervention(Base):
12     __tablename__ = "interventions"
13
14     id_intervention = Column(UUID(as_uuid=True), primary_key=True,
15                               default=uuid.uuid4)
16     vehicule_id     = Column(UUID(as_uuid=True), nullable=False)
17     vehicule_immat  = Column(String, nullable=False)
18     technicien_id   = Column(String, nullable=False)
19     type            = Column(String, nullable=False)
20     date_planifiee  = Column(DateTime, nullable=False)
21     date_realisation = Column(DateTime, nullable=True)
22     statut          = Column(Enum(StatutIntervention),
23                               default=StatutIntervention.PLANIFIEE)
24     cout            = Column(Float, nullable=True)
25     description     = Column(String, nullable=True)
26     created_at      = Column(DateTime, default=datetime.datetime.
27                               utcnow)
```

Listing 10.1 – Définition du modèle d'intervention (app/database/models.py)

Chapitre 11

API REST et Sécurité Keycloak (FastAPI)

11.1 Endpoints REST

lightblue Méth.	URL	Rôles requis	Description
GET	/api/interventions	tous	Lister les interventions
POST	/api/interventions	admin-fleet, technicien	Créer une intervention
GET	/api/interventions/:id	tous	Détail d'une intervention
PATCH	/api/interventions/:id/statut	admin-fleet, technicien	Mettre à jour le statut
DELETE	/api/interventions/:id	admin-fleet	Supprimer ou annuler
GET	/health	public	Liveness probe

TABLE 11.1 – Endpoints REST du maintenance-service

11.2 Sécurité JWT

```
1 from fastapi import Depends, HTTPException
2 from fastapi.security import HTTPBearer,
   HTTPAuthorizationCredentials
3 from jose import jwt
4 import urllib.request, json
5
6 security = HTTPBearer()
7 KEYCLOAK_CERTS_URL = (
8     "http://keycloak:8080/realms/fleet-management"
9     "/protocol/openid-connect/certs"
10 )
```

```
11
12 def verify_token(
13     credentials: HTTPAuthorizationCredentials = Depends(security)
14 ):
15     token = credentials.credentials
16     try:
17         with urllib.request.urlopen(KEYCLOAK_CERTS_URL) as url:
18             jwks = json.loads(url.read().decode())
19             payload = jwt.decode(token, jwks, algorithms=["RS256"],
20                                 audience="account")
21             roles = payload.get("realm_access", {}).get("roles", [])
22             if "technicien" not in roles and "admin-fleet" not in roles
23                 :
24                 raise HTTPException(status_code=403,
25                                     detail="Role insuffisant")
26             return payload
27     except Exception:
28         raise HTTPException(status_code=401,
29                             detail="Token invalide ou expire")
```

Listing 11.1 – Validation cryptographique du jeton (app/core/auth.py)

Chapitre 12

Intégration Apache Kafka (FastAPI)

12.1 Consommateur

```
1 from confluent_kafka import Consumer
2 import json
3 from app.database.database import SessionLocal
4 from app.database.models import VehiculeLocal
5
6 def start_kafka_consumer():
7     conf = {'bootstrap.servers': 'localhost:9092',
8            'group.id': 'maintenance-group'}
9     consumer = Consumer(conf)
10    consumer.subscribe(['fleet.vehicules.created',
11                       'fleet.vehicules.statut'])
12    while True:
13        msg = consumer.poll(1.0)
14        if msg is None or msg.error():
15            continue
16        topic = msg.topic()
17        data = json.loads(msg.value().decode('utf-8'))
18        db = SessionLocal()
19        if topic == 'fleet.vehicules.created':
20            nouveau_vehicule = VehiculeLocal(
21                id=data['id_vehicule'],
22                immatriculation=data['immatriculation']
23            )
24            db.add(nouveau_vehicule)
25            db.commit()
```

Listing 12.1 – Ingésion des véhicules (app/kafka/consumer.py)

Chapitre 13

Tests et Validation Client (FastAPI)

13.1 Validation de l'API avec Bruno

```
1 {
2   "vehicule_id":      "8bbbbd33-4f1e-4f1e-8e81-cdcdcdcdcdcd",
3   "vehicule_immat":   "XYZ-999",
4   "technicien_id":    "TECH-01",
5   "type":             "reparation",
6   "date_planifiee":   "2026-07-20T14:30:00Z",
7   "description":      "Panne de moteur signalee par Kafka"
8 }
```

Listing 13.1 – Payload d'entrée pour la création d'une intervention

```
1 {
2   "id_intervention":  "a42b3d48-f493-4895-b786-5e3d0d267095",
3   "vehicule_immat":   "XYZ-999",
4   "type":             "reparation",
5   "statut":           "planifiee",
6   "cout":             null,
7   "created_at":       "2026-03-31T12:25:11.212883"
8 }
```

Listing 13.2 – Réponse du serveur FastAPI (HTTP 201 Created)

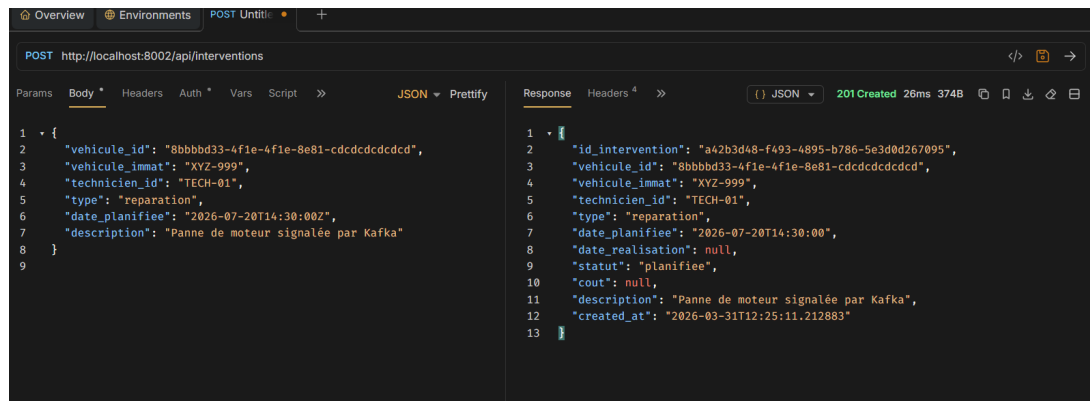


FIGURE 13.1 – Création d'une intervention validée via Bruno

13.2 Validation via l'API Gateway Apollo (GraphQL)

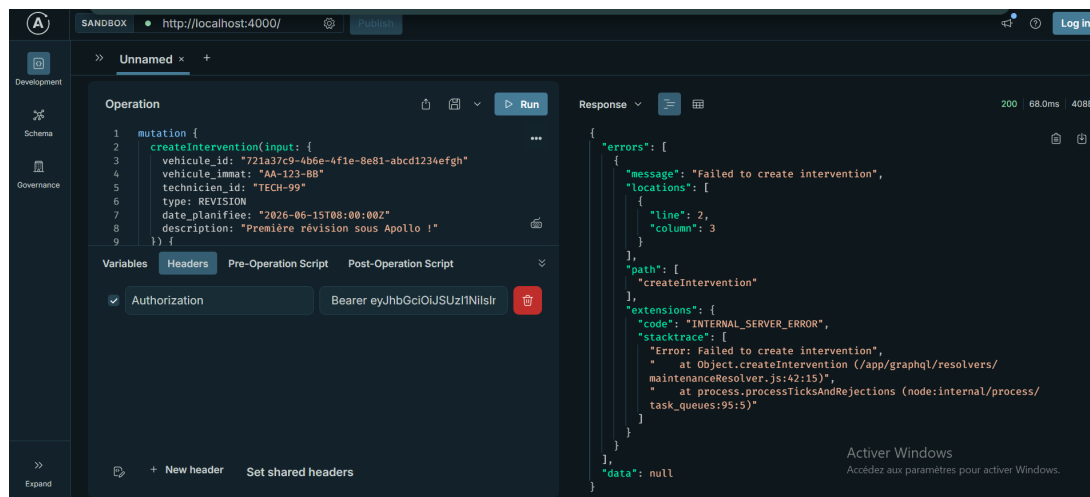


FIGURE 13.2 – Test d'une mutation GraphQL depuis Apollo Sandbox

13.3 Couverture des tests Pytest

```

===== test session starts =====
plugins: anyio-3.7.1, asyncio-0.21.1, cov-7.1.0
collected 11 items

test_main.py ..... [100%]

----- coverage: platform win32, python 3.11.8-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
app\__init__.py                     0      0   100%
app\controllers\routers.py         46      0   100%
app\core\auth.py                    22      4    81%   19-20, 26, 30
app\database\database.py            9      0   100%
app\database\models.py              20      0   100%
app\kafka\consumer.py               33     13    60%   32-34, ...
app\kafka\producer.py               16      4    75%   9, 15-17
app\main.py                          13      0   100%
app\schemas\schemas.py            22      0   100%
app\services\crud.py                62      0   100%
-----
TOTAL                               243     21    83%
===== 11 passed, 2 warnings in 7.45s =====

```

FIGURE 13.3 – Rapport d'exécution Pytest — couverture 83%

Conclusion

La réalisation simultanée du **service conducteurs en NestJS/TypeScript** et du **service maintenance en Python/FastAPI** valide l'agilité de notre système distribué hétérogène.

Service Conducteur (NestJS) :

- Architecture modulaire NestJS avec TypeORM et PostgreSQL.
- Saga Kafka choréographiée pour les assignations conducteur-véhicule.
- Sécurité JWT Keycloak avec RBAC 4 niveaux.
- Couverture Jest de plus **80%** sur 78 tests unitaires.

Service Maintenance (FastAPI) :

- Architecture Clean avec FastAPI, Pydantic et SQLAlchemy.
- Consumer Kafka asynchrone pour la synchronisation des véhicules.
- Sécurité Keycloak rigoureuse sur les endpoints.
- Couverture Pytest de **83%**, dépassant l'objectif de 80%.